Assignment 9: Feature Scaling and Model Evaluation

In [184…

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, mean_absolute_error, mean_squared_er
import numpy as np
import pandas as pd
import warnings

warnings.filterwarnings('ignore')
```

Q1: Feature Scaling Concepts

- **Standartization**: transforms freatures so they have a mean of 0 and standard deviation of 1
- **Min-Max Scaling**: rescales data to a fixed range, usually [0, 1]. It is calculated by subtracting the minimum value and dividing by range (min-max).
- **Robust scalling**: removes the median and scales the data according to the IQR.
- **Why feature scaling is important**: Feature scaling ensures that all features contribute equally to the model's preformance. Really important for models that calculate distances.

Q2: Load and Explore Dataset

In [185…

```
data = load_breast_cancer()

df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

X = df[data.feature_names]
y = df["target"]

print("Shape: ", {df.shape})
df.head()
```

```
Shape:  {(569, 31)}
```

Out[185…

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	s
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	

5 rows × 31 columns

Q3: Train-Test Split

In [186…

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

Q4: Apply Feature Scaling

Apply:

- StandardScaler
- MinMaxScaler
- RobustScaler

In [187…

```
std_scaler = StandardScaler()
X_train_std = std_scaler.fit_transform(X_train)
X_test_std = std_scaler.transform(X_test)

mm_scaler = MinMaxScaler()
X_train_mm = mm_scaler.fit_transform(X_train)
X_test_mm = mm_scaler.transform(X_test)

rb_scaler = RobustScaler()
X_train_rb = rb_scaler.fit_transform(X_train)
X_test_rb = rb_scaler.transform(X_test)
```

Q5: Train Models

Train Logistic Regression on:

1. Unscaled data
2. Standardized scaled data
3. MinMax scaled data
4. Robust Scaled data

In [188…

```
model = LogisticRegression()
model_std = LogisticRegression()
```

```
model_mm = LogisticRegression()
model_rb = LogisticRegression()
```

In [189…

```
# Train unscaled data
model.fit(X_train, y_train)
```

Out[189…

```
▾ LogisticRegression
▶ Parameters
```

In [190…

```
# Train stamdartozed scaled data
model_std.fit(X_train_std, y_train)
```

Out[190…

```
▾ LogisticRegression
▶ Parameters
```

In [191…

```
# Train minmax scaled data
model_mm.fit(X_train_mm, y_train)
```

Out[191…

```
▾ LogisticRegression
▶ Parameters
```

In [192…

```
# Train robust scaled data
model_rb.fit(X_train_rb, y_train)
```

Out[192…

```
▾ LogisticRegression
▶ Parameters
```

Q6: Evaluate Each Model

Evaluate each model using the corresponding test set and print the following:

- Accuracy
- MAE
- RMSE

In [193…

```
# For unscaled data
pred = model.predict(X_test)
acc = accuracy_score(y_test, pred)
mae = mean_absolute_error(y_test, pred)
rmse = np.sqrt(mean_squared_error(y_test, pred))
print(acc, mae, rmse)
```

```
0.956140350877193 0.043859649122807015 0.20942695414584775
```

In [194…

```
# For standartized scaled data
pred_std = model_std.predict(X_test_std)
acc_std = accuracy_score(y_test, pred_std)
```

```
mae_std = mean_absolute_error(y_test, pred_std)
rmse_std = np.sqrt(mean_squared_error(y_test, pred_std))
print(acc_std, mae_std, rmse_std)
```

```
0.9824561403508771 0.017543859649122806 0.13245323570650439
```

In [195…

```
# For minmax scaled data
pred_mm = model_mm.predict(X_test_mm)
acc_mm = accuracy_score(y_test, pred_mm)
mae_mm = mean_absolute_error(y_test, pred_mm)
rmse_mm = np.sqrt(mean_squared_error(y_test, pred_mm))
print(acc_mm, mae_mm, rmse_mm)
```

```
0.956140350877193 0.043859649122807015 0.20942695414584775
```

In [196…

```
# For robust scaled data
pred_rb = model_rb.predict(X_test_rb)
acc_rb = accuracy_score(y_test, pred_rb)
mae_rb = mean_absolute_error(y_test, pred_rb)
rmse_rb = np.sqrt(mean_squared_error(y_test, pred_rb))
print(acc_rb, mae_rb, rmse_rb)
```

```
0.9824561403508771 0.017543859649122806 0.13245323570650439
```

Q7: Write down the results in a table (use markdown syntax for that). Use the following example as a guide

Scaler type	Accuracy	MAE	RMSE
No Scaling	0.956	0.044	0.209
STD scaling	0.982	0.018	0.132
MinMax scaling	0.956	0.044	0.209
Robust scaling	0.982	0.018	0.132

Q8: Reflection

- Scaling improves the preformance and accuracy of the model, that calculates distances.
- Scaling is nessesary when features has different scales

In [ ]:

```

```